



**Instituto Tecnológico de Costa Rica**

Escuela de Ingeniería en computadores

CE-1104 - Fundamentos de sistemas computacionales

Grupo 4

Documento Técnico – Proyecto 1: StrangerTEC Morse Translator

Repositorio Github: <https://github.com/5amz/Proyecto-Funda-StrangerTEC.git>

**Profesor:**

Santiago Gamboa Ramírez

**Estudiantes:**

Samuel Ugalde Abrahams - 2026006212

Jacky Yin Lu – 2026006278

**Fechas:**

Creación de documento: 07/05/2026

Entrega: 09/05/2026

## Introducción

La idea del proyecto StrangerTEC Morse Translator es desarrollar un sistema interactivo inspirado en la serie *Stranger Things*, enfocado en la comunicación mediante luces y código Morse. La idea principal del proyecto es crear una maqueta conectada a una interfaz gráfica en computadora, donde dos jugadores puedan enviar, recibir e interpretar mensajes usando luces (leds), sonido (buzzer) y entradas físicas o digitales (pulsador/boton de teclado). El sistema utiliza una Raspberry Pi Pico W como sistema empujado, una interfaz digital de Python utilizando Tkinter, LEDs, pulsador, buzzer, un dipswitch y registros de corrimiento para controlar el panel de luces (Instituto Tecnológico de Costa Rica, 2026).

El código Morse se utiliza como base de comunicación porque permite representar letras y números mediante combinaciones de puntos y rayas (G.S.Pau, 2023). Estas señales se interpretarán según la duración de la presión del pulsador: una presión corta representa un punto y una presión larga representa una raya. Además, el sistema debe respetar pausas entre símbolos, letras y palabras para que el mensaje pueda ser entendido correctamente.

El sistema tiene dos modos principales de juego: el modo local, donde dos jugadores utilizan una misma maqueta y la computadora mediante comunicación USB, y el modo versus, donde dos maquetas se conectan por WiFi usando la Raspberry Pi Pico W. En ambos casos, el objetivo es que los jugadores interpreten o transmitan frases en código Morse y que el sistema compare sus respuestas con la frase original para asignar un puntaje. \

## Conclusiones

El proyecto permite aplicar conceptos importantes de sistemas computacionales, programación y electrónica básica, ya que combina una interfaz gráfica en Python con un sistema empotrado controlado por una Raspberry Pi Pico W. Esto hace que el proyecto no sea solamente un programa digital, sino un sistema completo donde el software y el hardware trabajan juntos.

El uso del código Morse como lenguaje de juego es adecuado, porque permite representar mensajes de forma clara usando señales simples. Las luces, el buzzer y el botón pulsador ayudan a que el usuario pueda relacionar directamente los puntos, rayas y pausas con el mensaje que debe interpretar o transmitir, además, el uso de luces y sonido a la permite retroalimentación visual y auditiva.

Finalmente, el proyecto permite practicar el trabajo por módulos, ya que se deben separar partes como la interfaz gráfica, la comunicación con la Raspberry Pi Pico W, la lectura del botón, el control de LEDs, el buzzer, la selección de frases y el sistema de puntaje, sin olvidar la conexión entre lo anterior y el sistema empotrado.

Por último, se ha de destacar lo importante que fue el trabajo en equipo durante el desarrollo del proyecto. La división de las tareas enfocado en las mejores cualidades (afinidad a la electrónica y afinidad a la programación) de cada integrante hizo que el proyecto fuera llevadero y permitió un buen resultado final. También es importante dividirse las tareas de una forma adecuada, para no dejar la mayoría de la carga sobre uno de los integrantes.

## Recomendaciones

Para la creación de la maqueta, se recomienda probar que cada uno de los componentes funcione adecuadamente antes de soldar, como el pulsador, LEDs, buzzer, el regulador de voltaje y el registro de corrimiento, si se suelda algún componente defectuoso, es difícil removerlo. Además, es recomendable utilizar cabeceras hembra para la conexión del Raspberry Pi Pico W y un socket para conectar el registro de corrimiento.

Es recomendable leer la documentación asociada a la Raspberry Pi Pico W y el registro de corrimiento. Esto sirve para poder saber la posición de cada pin y entender lo que permite hacer ese pin. Esto es importante sobre todo en estos componentes ya que, al tener tantos pines, es importante conectarlos adecuadamente.

Además, es importante comprobar la continuidad entre los componentes, para esto se utiliza un multímetro. También funciona para hacer troubleshooting de discontinuidades en ciertos componentes. También es importante revisar que el regulador de voltaje funcione correctamente con el multímetro.

Se recomienda utilizar placas preperforadas como la mostrada en la figura 1, ya que, al tener varias pistas horizontales discontinuas, permite conectar muchos componentes sin tener que soldar cables de conexión. Se recomienda utilizar cables de cobre en vez de alambres de cobre, ya que los alambres con quebradizos, y pueden dañarse fácilmente.

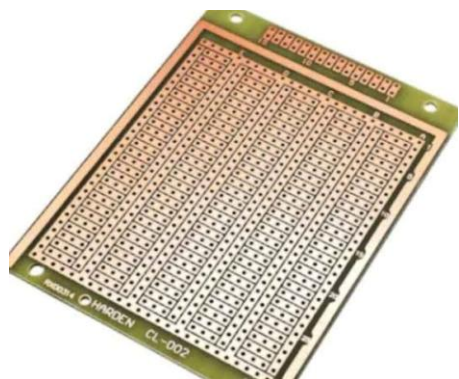


Figura 1. Placa preperforada.

Para la parte programada se recomienda realizar avances con objetivos claros en vez de un solo avance muy grande. Para esto se pueden usar los commits de github y enfocar cada commit en un objetivo que se quiera lograr. Además, se recomienda ir probando conforme se avanza el código para revisar que todo va funcionando bien. También, es importante comentar el código y usar nombres de variables que estén relacionados a su función para facilitar el entendimiento del código.

Para la interfaz, se recomienda evitar operaciones que bloquean esta misma, ya que pueden congelar la ventana hasta que se realicen. Por esta razón se utilizan hilos para manejar la comunicación serial y WiFi de manera independiente sin congelar el juego.

A la hora de corregir errores, es de utilidad utilizar prints como mensajes de depuración, especialmente en la comunicación entre la computadora y la Raspberry Pi pico W. Esto para verificar que esté ocurriendo las acciones deseadas durante la ejecución del código.

## **Análisis de resultados**

El desarrollo del proyecto permite comprobar que la comunicación mediante código Morse puede implementarse usando entradas digitales, luces y sonido. El sistema cumple con la idea principal del proyecto al permitir que una frase sea representada mediante señales Morse y que el jugador intente interpretarla o transmitirla.

Uno de los resultados principales es que la Raspberry Pi Pico W funciona como el centro de control de la maqueta. Esta recibe o envía información, controla el encendido de los LEDs, interpreta la entrada del botón y activa el buzzer durante la presión. Esto demuestra que el sistema empotrado puede responder a entradas físicas y generar salidas visibles y sonoras.

La interfaz gráfica en la computadora cumple una función importante porque permite configurar el juego, seleccionar o mostrar frases, visualizar puntajes y presentar los resultados finales. Según las instrucciones, la aplicación debe contener una lista de diez frases modificables, seleccionar frases aleatorias y mostrar retroalimentación de los puntajes de los jugadores.

El uso del botón pulsador representa uno de los retos más importantes del proyecto. Para que el sistema funcione correctamente, el programa debe medir cuánto tiempo se mantiene presionado el botón y decidir si corresponde a un punto o a una raya. También debe reconocer las pausas entre símbolos y letras. Si estos tiempos no se controlan bien, el mensaje puede quedar mal interpretado.

El buzzer también aporta al resultado final porque brinda retroalimentación auditiva mientras el usuario presiona el botón. Esto ayuda a que el jugador tenga una mejor referencia de la duración de cada señal. Además, las luces del panel permiten observar de forma visual las letras o números asociados al mensaje.

En general, el sistema demuestra una integración funcional entre hardware y software. El proyecto no se limita a traducir texto a Morse, sino que crea una dinámica de juego donde los jugadores compiten, reciben puntajes y deben usar correctamente la lógica del código Morse.

### Explicación de bibliotecas

Tabla I. Explicación de bibliotecas utilizadas.

| Biblioteca | Plataforma | Función principal en el proyecto                                                                                                                                                                                                                 |
|------------|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Tkinter    | Python     | Módulo para interfaces gráficas, construye ventanas, botones, etiquetas, cuadros de texto y pantallas para mostrar el juego (Python Software Foundation, 2024).                                                                                  |
| random     | Python     | Módulo para escogencias pseudoaleatorios. Utilizado para seleccionar frases de forma psuedoaleatoria de la lista en cada ronda del juego (Python Software Foundation, 2024).                                                                     |
| time       | Python     | Módulo que permite obtener el tiempo real y realizar pausas en la ejecución. Se utiliza medir duraciones y pausas en la pulsación para diferenciar punto, raya, separación entre símbolos, letras y palabras (Python Software Foundation, 2024). |
| threading  | Python     | Módulo para ejecutar varios hilos a la vez. Utilizado para leer el puerto serial sin bloquear la interfaz principal (Python Software Foundation, 2024).                                                                                          |
| serial     | Python     | Biblioteca pySerial que facilita el acceso al puerto serie. Permitir la comunicación entre la PC y la Raspberry Pi Pico W a través del puerto USB/serial (pySerial — pySerial 3.5 Documentation, s.f.).                                          |

|                         |                        |                                                                                                                                                                                                                                                           |
|-------------------------|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| serial.tools.list_ports | Python                 | Submódulo de pySerial que lista y enumera todos los puertos serie disponibles en el sistema. Utilizado para conectar a la maqueta automáticamente (Tools — pySerial 3.5 Documentation, s.f.).                                                             |
| socket                  | Python/<br>MicroPython | Módulo que crea un canal de comunicación para transmitir y recibir datos entre dispositivos a través de una red. Se utiliza para que La Raspberry pueda comunicarse con la computadora (Socket – Socket Module — MicroPython Latest Documentation, s.f.). |
| network                 | MicroPython            | Módulo que permite configurar la interfaz inalámbrica del microcontrolador para conectarlo a una red WiFi. Se utiliza para conectar el Raspberry al WiFi (network – network configuration — MicroPython Latest Documentation, s.f.)                       |
| machine                 | MicroPython            | Módulo de MicroPython que permite controlar el hardware del microcontrolador. Se utiliza para Controlar pines GPIO, PWM y componentes físicos (boton, LEDs y buzzer) (class Pin – control I/O pins — MicroPython Latest Documentation, s.f.).             |
| utime                   | MicroPython            | Versión para MicroPython de time, para sistemas empujados. Se utiliza para interpretar el tiempo de pulsación de las señales Morse en el microcontrolador (Time – Time Related Functions — MicroPython Latest Documentation, s.f.).                       |
| sys                     | MicroPython            | Módulo que da acceso a variables y funciones que interactúan con el intérprete de MicroPython. Se utiliza para ver si llegan datos de la computadora (Python Software Foundation, 2024).                                                                  |



|        |             |                                                                                                                                                                                                        |
|--------|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| select | MicroPython | Módulo de MicroPython que permite detectar entradas y salidas simultáneamente sin bloquear la ejecución. Utilizado para detectar si entran datos de la computadora (Python Software Foundation, 2024). |
|--------|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### Diagrama de arquitectura básica

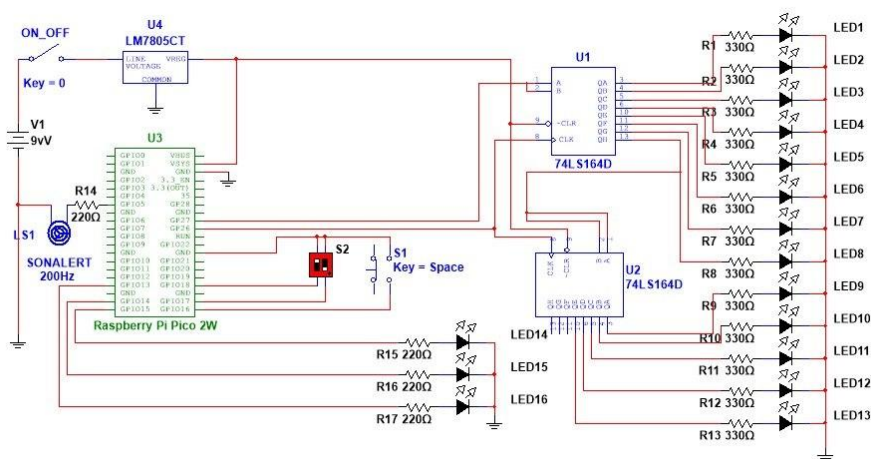


Figura 2. Diagrama detallado de sistema de empotrado.

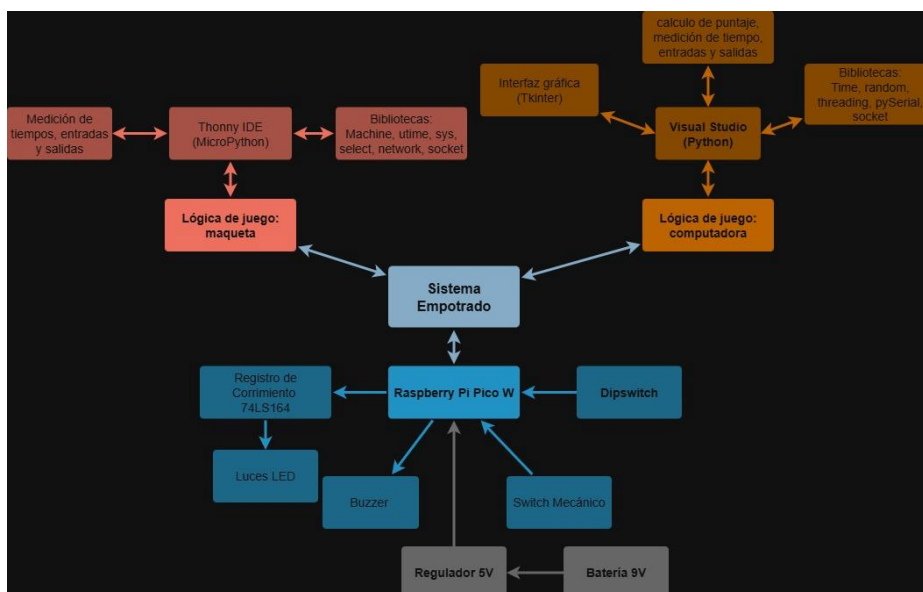


Figura 3. Diagrama de arquitectura básica

## Diagrama de módulos del sistema

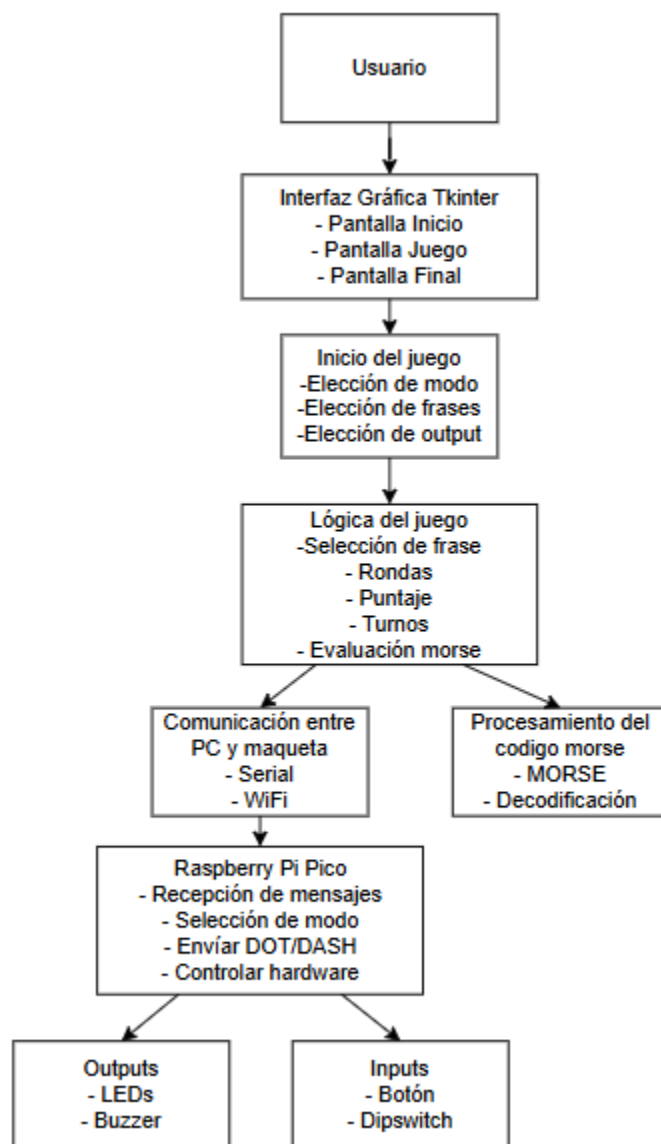


Figura 4. Diagrama de módulos del sistema

## Referencias

class Pin – control I/O pins — MicroPython latest documentation. (s.f.).

<https://docs.micropython.org/en/latest/library/machine.Pin.html>

G.S.Pau. (2023, August 16). *Traductor de Código Morse | Códigos Secretos | G.S.Pau.*

<https://www.gspau.es/traductor-morse/>

Instituto Tecnológico de Costa Rica. (2026). *StrangerTEC Morse Translator: Proyecto I.*

Escuela de Ingeniería en Computadores, CE-1104 Fundamentos de Sistemas  
Computacionales.

network — network configuration — MicroPython latest documentation. (s.f.).

<https://docs.micropython.org/en/latest/library/network.html>

Python Software Foundation. (2024). random — Generate pseudo-random numbers. Python 3

Documentation. <https://docs.python.org/3/library/random.html>

Python Software Foundation. (2024). select — Waiting for I/O completions. Python 3

Documentation. <https://docs.python.org/3/library/select.html>

Python Software Foundation. (2024). sys — System-specific parameters and functions. Python 3

Documentation. <https://docs.python.org/3/library/sys.html>

Python Software Foundation. (2024). threading — Thread-based parallelism. Python 3

Documentation. <https://docs.python.org/3/library/threading.html>

Python Software Foundation. (2024). time — Time access and conversions. Python 3

Documentation. <https://docs.python.org/3/library/time.html>

Python Software Foundation. (2024). tkinter — Python interface to Tcl/Tk. Python 3

Documentation. <https://docs.python.org/3/library/tkinter.html>

pySerial — pySerial 3.5 documentation. (s.f.).

<https://pyserial.readthedocs.io/en/latest/pyserial.html>

socket – socket module — MicroPython latest documentation. (s.f.).

<https://docs.micropython.org/en/latest/library/socket.html>

time – time related functions — MicroPython latest documentation. (s.f.).

<https://docs.micropython.org/en/latest/library/time.html>

Tools — pySerial 3.5 documentation. (s.f.). <https://pyserial.readthedocs.io/en/latest/tools.html>